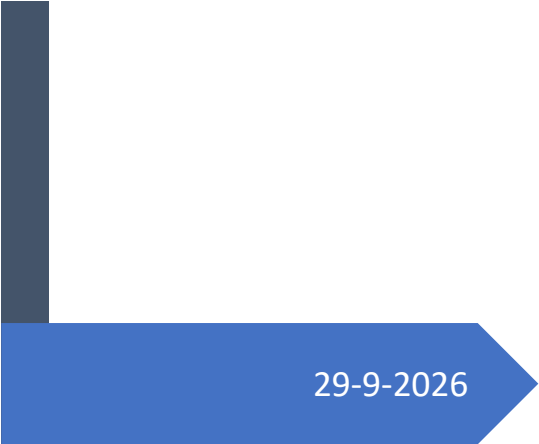
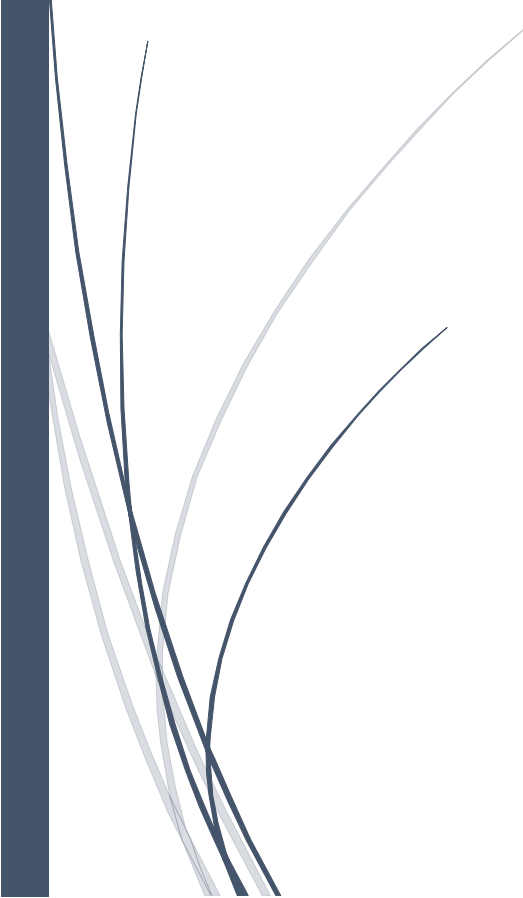


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

29-9-2026

Laboratorio 2

Haskell

Several thin, curved lines in dark blue and light grey originate from the bottom left and curve upwards and to the right.

Ana Sofia Caballero Serrano
Maria Victoria Barragan Avila
Paula Sofia Hurtado Nagles

Ejercicio 1 - Manejo de listas

+ ✂ 📄 📋 ▶ ■ ↺ ▶ Code ▼

```
[2]: milength [] = 0
      milength (x:xs) = 1 + milength xs
      milength [4,7,2,9]
```

Use foldr

Found:

```
milength [] = 0
milength (x : xs) = 1 + milength xs
4
```

Why Not:

```
milength xs = foldr (\ x -> (+) 1) 0 xs
```

```
[3]: milast :: [a] -> a
      milast [x] = x
      milast (x:xs) = milast xs
      milast [10,20,30,40]
```

```
40
```

```
[4]: duplicar :: [Int] -> [Int]
      duplicar [] = []
      duplicar (x:xs) = x:x:duplicar xs
      duplicar [2,4,6]
```

```
[2,2,4,4,6,6]
```

```
[5]: eliminarPares :: [Int] -> [Int]
      eliminarPares [] = []
      eliminarPares (x:xs)
        | even x = eliminarPares xs
        | otherwise = x : eliminarPares xs
      eliminarPares [1,2,3,4,5,6]
```

```
[1,3,5]
```

Ejercicio 2 - Listas por comprensión

+ ✂ 📄 📌 ▶ ■ ↺ ▶▶ Code ▼

```
[7]: cuadrados :: [Int]
cuadrados = [x^2 | x <- [1..10]]
cuadrados
```

[1,4,9,16,25,36,49,64,81,100]

```
[9]: divisibles3 :: [Int]
divisibles3 = [x | x <- [1..30], mod x 3 == 0]
divisibles3
```

[3,6,9,12,15,18,21,24,27,30]

```
[10]: pares :: [(Int, Int)]
pares = [(x,y) | x <- [1..5], y <- [1..5], x < y]
pares
```

[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]

```
[12]: divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], mod n x == 0]
divisores 12
```

[1,2,3,4,6,12]

```
[13]: divisoresPropios :: Int -> [Int]
divisoresPropios n = [x | x <- [1..n-1], mod n x == 0]
divisoresPropios 12
```

[1,2,3,4,6]

```
[16]: esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n
esPerfecto 6
```

True

Ejercicio 3 - Funciones de orden superior

-- 1a. Suma de todos los elementos

sumaLista :: [Int] -> Int

sumaLista = foldr (+) 0

-- 1b. Contar los mayores que 10

mayoresQue10 :: [Int] -> Int

mayoresQue10 = length . filter (> 10)

-- 1c. Producto de la lista

productoLista :: [Int] -> Int

productoLista = foldr (*) 1

-- 1d. Celsius a Fahrenheit

celsiusAFahrenheit :: [Double] -> [Double]

celsiusAFahrenheit = map (\c -> c * 9 / 5 + 32)

-- 1e. Suma 5 a los impares, deja igual los pares

ajustarImpares :: [Int] -> [Int]

ajustarImpares = map (\x -> if odd x then x + 5 else x)

Defina las siguientes funciones con filter

-- 2a. Nombres con más de 5 letras

nombresLargos :: [String] -> [String]

nombresLargos = filter ((> 5) . length)

-- 2b. Números negativos

soloNegativos :: [Int] -> [Int]

```
soloNegativos = filter (< 0)
```

```
-- 2c. Edades entre 18 y 25
```

```
edadesUniversitarios :: [Int] -> [Int]
```

```
edadesUniversitarios = filter (\e -> e >= 18 && e <= 25)
```

defina las siguientes funciones usando filter y map

```
-- 3a. IVA (19%) solo a valores >= 1000
```

```
agregarIVA :: [Double] -> [Double]
```

```
agregarIVA = map (* 1.19) . filter (>= 1000)
```

```
-- 3b. Palabras que empiezan con vocal -> su longitud
```

```
longitudesVocales :: [String] -> [Int]
```

```
longitudesVocales = map length . filter empiezaConVocal
```

```
where
```

```
    empiezaConVocal p = not (null p) && toLower (head p) `elem` "aeiou"
```

```
-- 3c. Cuadrados de los pares
```

```
cuadradosPares :: [Int] -> [Int]
```

```
cuadradosPares = map (^ 2) . filter even
```

defina las siguientes funciones usando any o all

```
-- 4a. ¿Hay algún negativo?
```

```
hayNegativo :: [Int] -> Bool
```

```
hayNegativo = any (< 0)
```

-- 4b. ¿Todos son pares?

todosPares :: [Int] -> Bool

todosPares = all even

-- 4c. ¿Todos empiezan con mayúscula?

nombresCorrectos :: [String] -> Bool

nombresCorrectos = all (\n -> not (null n) && isUpper (head n))

defina las siguientes funciones con takeWhile o dropWhile

-- 5a. Elementos menores que 100 (lista ordenada)

menoresQue100 :: [Int] -> [Int]

menoresQue100 = takeWhile (< 100)

-- 5b. Quitar ceros iniciales

sinCerosIniciales :: [Int] -> [Int]

sinCerosIniciales = dropWhile (== 0)

Ejercicio 4 - Funciones que reciben y retornan funciones

-- 1. Recibe una función que transforma un dato y la aplica a una lista

```
procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
```

```
procesarLecturas f xs = map f xs
```

```
celsiusAFahrenheit :: Double -> Double
```

```
celsiusAFahrenheit c = c * 9 / 5 + 32
```

-- 2. Retorna un filtro personalizado

```
crearFiltro :: (a -> Bool) -> ([a] -> [a])
```

```
crearFiltro p = filter p
```

-- 3. Genera verificadores de rango

```
enRango :: Int -> Int -> (Int -> Bool)
```

```
enRango minimo maximo = \x -> x >= minimo && x <= maximo
```

-- 4. Eleva a una potencia dada

```
potenciador :: Int -> (Int -> Int)
```

```
potenciador n = \x -> x ^ n
```

Ejercicio 5 - “El problema de las dos jarras” usando A*

1. Funciones de transición

haskell

transferirAB :: Estado -> Estado

transferirAB (a, b) = (a - t, b + t)

where t = min a (capB - b)

transferirBA :: Estado -> Estado

transferirBA (a, b) = (a + t, b - t)

where t = min b (capA - a)

2. Heurística admisible

haskell

heuristica :: Estado -> Costo

heuristica e = if e == (4, 0) then 0 else 1

Es admisible porque nunca sobreestima: en la meta faltan 0 pasos ($h = 0$), y en cualquier otro estado falta al menos 1 acción ($h = 1$).

3. Resultado de A*

aStar vecinos heuristica (0,0) (4,0) termina en ((4,0),7), es decir, la meta con costo 7.

Secuencia óptima (7 acciones):

$(0,0) \rightarrow (5,0) \rightarrow (2,3) \rightarrow (2,0) \rightarrow (0,2) \rightarrow (5,2) \rightarrow (4,3) \rightarrow (4,0)$

Llenar A, verter A→B, vaciar B, verter A→B, llenar A, verter A→B, vaciar B.

Respuesta final: se necesitan **7 pasos** para llegar al estado (4, 0).

Retrospectiva

1. Tiempo total invertido (horas/hombre)

Integrante Laboratorio Fuera del lab. Total

Ana	2 h	20 min	2 h 20 min
Sofía	2 h	20 min	2 h 20 min
Victoria	2 h	20 min	2 h 20 min
Equipo	6 h	1 h	7 h

2. Estado actual del laboratorio

Completo. Los ejercicios se desarrollaron durante la sesión de 3 horas, de las cuales trabajamos efectivamente cerca de 2 h, descontando distracciones. Después del laboratorio solo invertimos unos 20 minutos por integrante en organizar el documento final.

3. Mayor logro

Haber desarrollado todos los ejercicios aplicando lo visto en clase. Nos mostró que podemos llevar la teoría a soluciones que funcionan.

4. Mayor problema técnico

Entender que el aStar base devuelve la lista de nodos expandidos y no el camino, además de errores de sintaxis y tipos en Haskell. Lo resolvimos revisando los apuntes de clase, probando en el intérprete paso a paso y agregando una versión que guarda el camino dentro de cada nodo.

5. Qué hicimos bien y qué mejoraremos

Bien: nos dividimos el trabajo entre las tres y todas participamos. **Mejorar:** evitar distracciones durante la sesión, repasar los temas antes de la clase, leer el enunciado completo desde el inicio y reservar los últimos minutos para revisar y probar antes de entregar.

6. Referencias

Escuela Colombiana de Ingeniería Julio Garavito. (2026). *Apuntes y diapositivas de clase* [Material de clase].